

RV64 — Skipped Extensions Decoding Reference

Extensions deliberately excluded from 5-stage pipeline implementation to reduce complexity. Kept for reference — to know exactly what you are not implementing.

opcode	rd	funct3	rs1	rs2	funct7/imm	aq/rl	imm
--------	----	--------	-----	-----	------------	-------	-----

A — Atomic Extension

Why skipped: LR/SC need a reservation set (extra state) and the MEM stage must do an atomic read-modify-write — impossible without bus locking or cache coherence. AMO instructions cannot be split across pipeline stages safely.

Mnemonic	Fmt	opcode	funct3	funct7 / imm	Bit layout [31:0]	Operation	Complexity reason
A — Atomic Extension (RV64A) WHY SKIPPED: requires bus locking / cache coherence, breaks simple pipeline memory stage							
A — Load-Reserved / Store-Conditional (opcode 0101111)							
LR.W	A	0101111	010	00010 aq rl 00000	f5aqrlrs2rs1f3rd0101111	rd=Mem[rs1]; reserve address (32-bit)	Sets reservation; SC must check — needs memory bus lock
LR.D	A	0101111	011	00010 aq rl 00000	f5aqrlrs2rs1f3rd0101111	rd=Mem[rs1]; reserve address (64-bit)	64-bit version — same bus-lock requirement
SC.W	A	0101111	010	00011 aq rl rs2	f5aqrlrs2rs1f3rd0101111	if reserved: Mem[rs1]=rs2; rd=0 else rd=1	Conditional store — reservation check in MEM stage
SC.D							
A — Atomic Memory Operations (opcode 0101111, funct3=010/011 for W/D)							
AMOSWAP.W	A	0101111	010	00001 aq rl	f5aqrlrs2rs1f3rd0101111	tmp=Mem[rs1]; Mem[rs1]=rs2; rd=tmp	Read-modify-write; needs atomic MEM access
AMOSWAP.D	A	0101111	011	00001 aq rl	f5aqrlrs2rs1f3rd0101111	64-bit atomic swap	Same — 64-bit
AMOADD.W	A	0101111	010	00000 aq rl	f5aqrlrs2rs1f3rd0101111	Mem[rs1] += rs2; rd=old value (32-bit)	Atomic add — all AMOs stall pipeline in MEM
AMOADD.D	A	0101111	011	00000 aq rl	f5aqrlrs2rs1f3rd0101111	Mem[rs1] += rs2; rd=old value (64-bit)	64-bit atomic add
AMOAND.W	A	0101111	010	01100 aq rl	f5aqrlrs2rs1f3rd0101111	Mem[rs1] &= rs2; rd=old (32-bit)	Atomic AND
AMOAND.D	A	0101111	011	01100 aq rl	f5aqrlrs2rs1f3rd0101111	Mem[rs1] &= rs2; rd=old (64-bit)	64-bit atomic AND
AMOOR.W	A	0101111	010	01000 aq rl	f5aqrlrs2rs1f3rd0101111	Mem[rs1] = rs2; rd=old (32-bit)	Atomic OR
AMOOR.D	A	0101111	011	01000 aq rl	f5aqrlrs2rs1f3rd0101111	Mem[rs1] = rs2; rd=old (64-bit)	64-bit atomic OR
AMOXOR.W	A	0101111	010	00100 aq rl	f5aqrlrs2rs1f3rd0101111	Mem[rs1] ^= rs2; rd=old (32-bit)	Atomic XOR
AMOXOR.D	A	0101111	011	00100 aq rl	f5aqrlrs2rs1f3rd0101111	Mem[rs1] ^= rs2; rd=old (64-bit)	64-bit atomic XOR
AMOMIN.W	A	0101111	010	10000 aq rl	f5aqrlrs2rs1f3rd0101111	Mem[rs1]=min(Mem,rs2) signed; rd=old	Atomic signed min

Mnemonic	Fmt	opcode	funct3	funct7 / imm	Bit layout [31:0]	Operation	Complexity reason
AMOMIN.D	A	0101111	011	10000 aq rl	f5aqrlrs2rs1f3rd0101111	64-bit atomic signed min	64-bit
AMOMAX.W	A	0101111	010	10100 aq rl	f5aqrlrs2rs1f3rd0101111	Mem[rs1]=max(Mem,rs2) signed; rd=old	Atomic signed max
AMOMAX.D	A	0101111	011	10100 aq rl	f5aqrlrs2rs1f3rd0101111	64-bit atomic signed max	64-bit
AMOMINU.W	A	0101111	010	11000 aq rl	f5aqrlrs2rs1f3rd0101111	Mem[rs1]=min(Mem,rs2) unsigned; rd=old	Atomic unsigned min
AMOMINU.D	A	0101111	011	11000 aq rl	f5aqrlrs2rs1f3rd0101111	64-bit atomic unsigned min	64-bit
AMOMAXU.W	A	0101111	010	11100 aq rl	f5aqrlrs2rs1f3rd0101111	Mem[rs1]=max(Mem,rs2) unsigned; rd=old	Atomic unsigned max
AMOMAXU.D	A	0101111	011	11100 aq rl	f5aqrlrs2rs1f3rd0101111	64-bit atomic unsigned max	64-bit

F/D — Single & Double Precision Floating Point

Why skipped: Requires a completely separate FP register file (f0–f31, each 64-bit), FPU in EX stage, FP-specific hazard detection, and cross-file forwarding (FMV instructions cross between int and FP). The R4-type fused multiply-add (FMADD etc.) needs 3 source registers — a new forwarding path your pipeline doesn't have.

Mnemonic	Fmt	opcode	funct3	funct7 / imm	Bit layout [31:0]	Operation	Complexity reason
F/D — Single & Double Precision Float (RV64F + RV64D) WHY SKIPPED: separate FP register file (f0–f31), FPU datapath, FP hazard unit							
F — FP Loads / Stores (F=0000111/0100111, D=0000111/0100111 funct3 differs)							
FLW	I	0000111	010	offset	imm[11:0]rs1010rd0000111	fd=Mem[rs1+imm] (32-bit float)	Loads into FP reg file — separate from int pipeline
FLD	I	0000111	011	offset	imm[11:0]rs1011rd0000111	fd=Mem[rs1+imm] (64-bit double)	Double load — FP reg file
FSW	S	0100111	010	offset	im[11:5]rs2rs1010im[4:0]0100111	Mem[rs1+imm]=fs2 (32-bit)	FP store — rs2 from FP reg file
FSD							
F — FP Arithmetic (opcode 1010011, fmt=00 single / 01 double)							
FADD.S	R	1010011	rm	0000000	0000000rs2rs1f3rd1010011	fd=fs1+fs2 (single)	FPU add — multicycle EX, separate hazard tracking
FADD.D	R	1010011	rm	0000001	0000001rs2rs1f3rd1010011	fd=fs1+fs2 (double)	Double precision add
FSUB.S	R	1010011	rm	0000100	0000100rs2rs1f3rd1010011	fd=fs1-fs2 (single)	FPU subtract
FSUB.D	R	1010011	rm	0000101	0000101rs2rs1f3rd1010011	fd=fs1-fs2 (double)	Double subtract
FMUL.S	R	1010011	rm	0001000	0001000rs2rs1f3rd1010011	fd=fs1*fs2 (single)	FPU multiply
FMUL.D	R	1010011	rm	0001001	0001001rs2rs1f3rd1010011	fd=fs1*fs2 (double)	Double multiply
FDIV.S	R	1010011	rm	0001100	0001100rs2rs1f3rd1010011	fd=fs1/fs2 (single)	FPU divide — very long latency
FDIV.D							
FSQRT.S	R	1010011	rm	0101100	0101100rs2rs1f3rd1010011	fd=sqrt(fs1) (single)	Square root — long latency

Mnemonic	Fmt	opcode	funct3	funct7 / imm	Bit layout [31:0]	Operation	Complexity reason
FSQRT.D	R	1010011	rm	0101101	0101101 rs2 rs1 f3 rd 1010011	fd=sqrt(fs1) (double)	Double sqrt
F — Fused Multiply-Add (4-register R4-type — unique format)							
FMADD.S	R4	1000011	rm	rs3 00	rs3 fmt rs2 rs1 rm rd 1000011	fd=fs1*fs2+fs3 (single)	3 FP src regs — new forwarding paths needed
FMADD.D	R4	1000011	rm	rs3 01	rs3 fmt rs2 rs1 rm rd 1000011	fd=fs1*fs2+fs3 (double)	4-operand instruction
FMSUB.S	R4	1000111	rm	rs3 00	rs3 fmt rs2 rs1 rm rd 1000111	fd=fs1*fs2-fs3 (single)	Fused multiply-subtract
FMSUB.D	R4	1000111	rm	rs3 01	rs3 fmt rs2 rs1 rm rd 1000111	fd=fs1*fs2-fs3 (double)	Double
FNMADD.S	R4	1001111	rm	rs3 00	rs3 fmt rs2 rs1 rm rd 1001111	fd=-(fs1*fs2+fs3) (single)	Negated fused MAC
FNMADD.D							
FNMSUB.S	R4	1001011	rm	rs3 00	rs3 fmt rs2 rs1 rm rd 1001011	fd=-(fs1*fs2-fs3) (single)	Negated fused MS
FNMSUB.D	R4	1001011	rm	rs3 01	rs3 fmt rs2 rs1 rm rd 1001011	fd=-(fs1*fs2-fs3) (double)	Double
F/D — Compare, Convert, Move (opcode 1010011)							
FEQS.S	R	1010011	010	1010000	1010000 rs2 rs1 f3 rd 1010011	rd=(fs1==fs2)?1:0	FP compare — result into int reg
FEQS.D	R	1010011	010	1010001	1010001 rs2 rs1 f3 rd 1010011	rd=(fs1==fs2)?1:0 (double)	Double compare
FLTS.S	R	1010011	001	1010000	1010000 rs2 rs1 f3 rd 1010011	rd=(fs1<fs2)?1:0	FP less-than
FLTS.D	R	1010011	001	1010001	1010001 rs2 rs1 f3 rd 1010011	rd=(fs1<fs2)?1:0 (double)	Double LT
FLES.S	R	1010011	000	1010000	1010000 rs2 rs1 f3 rd 1010011	rd=(fs1<=fs2)?1:0	FP less-or-equal
FLES.D							
FCVT.W.S	R	1010011	rm	1100000 00000	1100000 rs2 rs1 f3 rd 1010011	rd=int(fs1) signed 32	FP->int conversion crosses reg files
FCVT.WU.S	R	1010011	rm	1100000 00001	1100000 rs2 rs1 f3 rd 1010011	rd=uint(fs1) 32-bit	FP->uint
FCVT.L.D	R	1010011	rm	1100001 00010	1100001 rs2 rs1 f3 rd 1010011	rd=int64(fd1)	Double->int64 (RV64D)
FCVT.S.W	R	1010011	rm	1101000 00000	1101000 rs2 rs1 f3 rd 1010011	fd=float(rs1) signed 32	int->FP crosses reg files
FCVT.D.L	R	1010011	rm	1101001 00010	1101001 rs2 rs1 f3 rd 1010011	fd=double(rs1) int64	int64->double (RV64D)
FMV.X.W	R	1010011	000	1110000 00000	1110000 rs2 rs1 f3 rd 1010011	rd=bitcast(fs1) 32-bit	Bit-move FP->int reg
FMV.W.X	R	1010011	000	1111000 00000	1111000 rs2 rs1 f3 rd 1010011	fd=bitcast(rs1) 32-bit	Bit-move int->FP reg
FMV.X.D	R	1010011	000	1110001 00000	1110001 rs2 rs1 f3 rd 1010011	rd=bitcast(fd1) 64-bit	Double bit-move FP->int (RV64D)
FMV.D.X	R	1010011	000	1111001 00000	1111001 rs2 rs1 f3 rd 1010011	fd=bitcast(rs1) 64-bit	Double bit-move int->FP (RV64D)

Mnemonic	Fmt	opcode	funct3	funct7 / imm	Bit layout [31:0]	Operation	Complexity reason
FCLASS.S	R	1010011	001	1110000 00000	1110000rs2rs1f3rd1010011	rd=class(fs1) bitmask	Classify FP (NaN, inf, zero, etc.)
FCLASS.D	R	1010011	001	1110001 00000	1110001rs2rs1f3rd1010011	rd=class(fd1) bitmask	Double classify
FSGNJ.S	R	1010011	000	0010000	0010000rs2rs1f3rd1010011	fd= fs1 with sign of fs2	FP sign inject
FSGNJ.D	R	1010011	000	0010001	0010001rs2rs1f3rd1010011	fd= fd1 with sign of fd2	Double sign inject
FMIN.S	R	1010011	000	0010100	0010100rs2rs1f3rd1010011	fd=min(fs1,fs2)	FP minimum
FMIN.D	R	1010011	000	0010101	0010101rs2rs1f3rd1010011	fd=min(fd1,fd2)	Double minimum
FMAX.S	R	1010011	001	0010100	0010100rs2rs1f3rd1010011	fd=max(fs1,fs2)	FP maximum
FMAX.D	R	1010011	001	0010101	0010101rs2rs1f3rd1010011	fd=max(fd1,fd2)	Double maximum

C — Compressed Extension (16-bit instructions)

Why skipped: Instructions are 16 or 32 bits — the fetch stage can no longer assume 4-byte alignment. You need an instruction buffer, a 16/32-bit detector, and PC logic that can advance by 2 or 4. C.JALR returns to PC+2 (not PC+4), breaking standard link-register assumptions. Only 8 registers (x8–x15) are addressable in most compressed formats.

Mnemonic	Fmt	opcode	funct3	funct7 / imm	Bit layout [31:0]	Operation	Complexity reason
C — Compressed Extension (16-bit instructions) WHY SKIPPED: variable length (16/32-bit mix) breaks IF stage — needs alignment buffer & PC realignment logic							
C — Quadrant 0 (op[1:0]=00) — Loads & Stores							
C.ADDI4SPN	CIW	00	000	nzuimm[9:2]	funct3imm[9:2]rd'op	rd'=sp+uimm (rd'=x8-x15)	Compressed imm add to sp
C.FLD	CL	00	001	uimm[5:3 7:6]	funct3immrs1'immrd'op	fd'=Mem[rs1'+uimm] (double)	FP load — also needs FP reg file
C.LW	CL	00	010	uimm[5:3 2:6]	funct3immrs1'immrd'op	rd'=Mem[rs1'+uimm] 32-bit	Compressed word load
C.LD	CL	00	011	uimm[5:3 7:6]	funct3immrs1'immrd'op	rd'=Mem[rs1'+uimm] 64-bit	Compressed dword load (RV64C)
C.FSD	CS	00	101	uimm[5:3 7:6]	funct3immrs1'immrs2'op	Mem[rs1'+uimm]=fs2' (double)	Compressed FP store
C.SW	CS	00	110	uimm[5:3 2:6]	funct3immrs1'immrs2'op	Mem[rs1'+uimm]=rs2' 32-bit	Compressed word store
C.SD	CS	00	111	uimm[5:3 7:6]	funct3immrs1'immrs2'op	Mem[rs1'+uimm]=rs2' 64-bit	Compressed dword store (RV64C)
C — Quadrant 1 (op[1:0]=01) — Branches, Jumps, Immediates							
C.NOP	CI	01	000	0	funct3immrd/rs1immop	NOP (ADDI x0,x0,0)	Expands to full NOP
C.ADDI	CI	01	000	nzimm[5 4:0]	funct3immrd/rs1immop	rd+=nzimm (rd!=x0)	Compressed add immediate
C.ADDIW	CI	01	001	imm[5 4:0]	funct3immrd/rs1immop	rd=sext(rd[31:0]+imm) (RV64C)	Compressed addiw
C.LI	CI	01	010	imm[5 4:0]	funct3immrd/rs1immop	rd=sext(imm)	Load immediate
C.ADDI16SP	CI	01	011	nzimm[9 4 6 8:7 5]	funct3immrd/rs1immop	sp+=nzimm*16	Stack pointer adjust
C.LUI	CI	01	011	nzimm[17 16:12]	funct3immrd/rs1immop	rd=nzimm<<12	Load upper imm (rd!=x2)
C.SRLI	CB	01	100	shamt[5 4:0]	funct3offrs1'offop	rd'=rd'>>shamt (logical)	Compressed shift right logical
C.SRAI	CB	01	100	shamt[5 4:0]	funct3offrs1'offop	rd'=rd'>>shamt (arithmetic)	Compressed shift right arith
C.ANDI	CB	01	100	imm[5 4:0]	funct3offrs1'offop	rd'=rd'&sext(imm)	Compressed AND immediate

Mnemonic	Fmt	opcode	funct3	funct7 / imm	Bit layout [31:0]	Operation	Complexity reason
C.SUB	CA	01	100	—	funct6 rd'/rs1' funct2 rs2' op	rd'=rd'-rs2'	Compressed subtract
C.XOR	CA	01	100	—	funct6 rd'/rs1' funct2 rs2' op	rd'=rd'^rs2'	Compressed XOR
C.OR	CA	01	100	—	funct6 rd'/rs1' funct2 rs2' op	rd'=rd' rs2'	Compressed OR
C.AND	CA	01	100	—	funct6 rd'/rs1' funct2 rs2' op	rd'=rd'&rs2'	Compressed AND
C.SUBW	CA	01	100	—	funct6 rd'/rs1' funct2 rs2' op	rd'=sext((rd'-rs2')[31:0]) RV64	Compressed subw
C.ADDW	CA	01	100	—	funct6 rd'/rs1' funct2 rs2' op	rd'=sext((rd'+rs2')[31:0]) RV64	Compressed addw
C.J	CJ	01	101	offset[11:1]	funct3 jump target[11:1] op	PC+=sext(offset<<1)	Compressed jump
C.BEQZ							
C.BNEZ	CB	01	111	offset[8 4:3 7:6 2:1 5]	funct3 off rs1' off op	if rd'!=0: PC+=sext(off)	Compressed branch NE zero
C — Quadrant 2 (op[1:0]=10) — Stack Loads/Stores, Register Ops							
C.SLLI	CI	10	000	shamt[5 4:0]	funct3 imm rd'/rs1 imm op	rd=rd<<shamt	Compressed shift left
C.FLDSP	CI	10	001	uimm[5 4:3 8:6]	funct3 imm rd'/rs1 imm op	fd=Mem[sp+uimm] double	FP stack load
C.LWSP	CI	10	010	uimm[5 4:2 7:6]	funct3 imm rd'/rs1 imm op	rd=Mem[sp+uimm] 32-bit	Stack-relative word load
C.LDSP	CI	10	011	uimm[5 4:3 8:6]	funct3 imm rd'/rs1 imm op	rd=Mem[sp+uimm] 64-bit (RV64C)	Stack-relative dword load
C.JR	CR	10	100	—	funct4 rd'/rs1 rs2 op	PC=rs1 (rs2=0, rd=0)	Compressed jump register
C.MV	CR	10	100	—	funct4 rd'/rs1 rs2 op	rd=rs2	Compressed move
C.EBREAK	CR	10	100	—	funct4 rd'/rs1 rs2 op	Breakpoint	Compressed ebreak
C.JALR	CR	10	100	—	funct4 rd'/rs1 rs2 op	ra=PC+2; PC=rs1	Compressed JALR — PC+2 not +4!
C.ADD	CR	10	100	—	funct4 rd'/rs1 rs2 op	rd=rd+rs2	Compressed add
C.FSDSP	CSS	10	101	uimm[5:3 8:6]	funct3 imm rs2 op	Mem[sp+uimm]=fs2 double	FP stack store
C.SWSP	CSS	10	110	uimm[5:2 7:6]	funct3 imm rs2 op	Mem[sp+uimm]=rs2 32-bit	Stack-relative word store
C.SDSP	CSS	10	111	uimm[5:3 8:6]	funct3 imm rs2 op	Mem[sp+uimm]=rs2 64-bit (RV64C)	Stack-relative dword store

V — Vector Extension (RVV 1.0)

Why skipped: Completely different execution model: vl (vector length) and vtype CSRs control how many elements each instruction processes. Needs a vector register file (32×VLEN bits), a multi-lane execution unit, and element-level masking. A single VLE64.V can transfer hundreds of bytes — the MEM stage cannot handle this as a single pipeline transaction. Incompatible with a simple 5-stage design without a complete redesign.

Mnemonic	Fmt	opcode	funct3	funct7 / imm	Bit layout [31:0]	Operation	Complexity reason
V — Vector Extension (RVV 1.0) WHY SKIPPED: completely different execution model — vector register file, vtype CSR, variable-length ops, not compatible with simple 5-stage pipeline							
V — Configuration (opcode 1010111)							
VSETVLI	I	1010111	111	zimm[10:0]	imm[11:0] rs1 111 rd 1010111	vl=min(rs1, vlmax); vtype=zimm	Sets vector length & type — no int equivalent
VSETIVLI	I	1010111	111	zimm uimm	imm[11:0] rs1 111 rd 1010111	vl=min(uimm, vlmax); vtype=zimm	Immediate vector length config
VSETVL	R	1010111	111	1xxxxxx	1xxxxxx rs2 rs1 f3 rd 1010111	vl=min(rs1,rs2[vlmax]); vtype=rs2	Register-based vtype
V — Vector Loads (opcode 0000111) — unit/strided/indexed variants							
VLE8.V	I	0000111	000	nf mew mop vm lumop	imm[11:0] rs1 000 rd 0000111	vd=Mem[rs1..] 8-bit elements	Loads entire vector reg — width depends on vl
VLE16.V	I	0000111	101	—	imm[11:0] rs1 101 rd 0000111	vd=Mem[rs1..] 16-bit elements	Variable count from vl CSR
VLE32.V	I	0000111	110	—	imm[11:0] rs1 110 rd 0000111	vd=Mem[rs1..] 32-bit elements	
VLE64.V							
VLSE64.V	I	0000111	111	stride	imm[11:0] rs1 111 rd 0000111	vd=strided Mem load 64-bit	Strided — rs2=byte stride
VLUXEI64.V	R	0000111	111	indexed	xxxxxxx rs2 rs1 f3 rd 0000111	vd=indexed gather load	Indexed — rs2=vector of offsets
V — Vector Stores (opcode 0100111)							
VSE8.V	S	0100111	000	—	im[11:5] rs2 rs1 000 im[4:0] 0100111	Mem[rs1..]=vs3 8-bit	Stores full vector register
VSE64.V							
VSSE64.V	S	0100111	111	stride	im[11:5] rs2 rs1 111 im[4:0] 0100111	strided store 64-bit	rs2=stride
V — Vector Arithmetic (opcode 1010111) — representative subset							
VADD.VV	R	1010111	000	000000 vm	000000 rs2 rs1 f3 rd 1010111	vd=vs2+vs1 (all elements)	Parallel add — vl elements processed
VADD.VX	R	1010111	100	000000 vm	000000 rs2 rs1 f3 rd 1010111	vd=vs2+rs1 (scalar broadcast)	Scalar broadcast add
VADD.VI	I	1010111	011	000000 vm	imm[11:0] rs1 011 rd 1010111	vd=vs2+imm	Immediate broadcast add
VSUB.VV	R	1010111	000	000010 vm	000010 rs2 rs1 f3 rd 1010111	vd=vs2-vs1	Vector subtract
VMUL.VV	R	1010111	010	100101 vm	100101 rs2 rs1 f3 rd 1010111	vd=vs2*vs1 (low half)	Vector multiply low
VDIV.VV	R	1010111	010	100001 vm	100001 rs2 rs1 f3 rd 1010111	vd=vs2/vs1 (signed)	Vector divide — very long latency
VAND.VV	R	1010111	000	001001 vm	001001 rs2 rs1 f3 rd 1010111	vd=vs2&vs1	Vector AND
VOR.VV	R	1010111	000	001010 vm	001010 rs2 rs1 f3 rd 1010111	vd=vs2 vs1	Vector OR
VXOR.VV	R	1010111	000	001011 vm	001011 rs2 rs1 f3 rd 1010111	vd=vs2^vs1	Vector XOR

Mnemonic	Fmt	opcode	funct3	funct7 / imm	Bit layout [31:0]	Operation	Complexity reason
VFADD.VV	R	1010111	001	000000 vm	000000rs2rs1f3rd1010111	vd=vs2+vs1 (float)	Vector FP add — needs FPU array
VFMUL.VV	R	1010111	001	100100 vm	100100rs2rs1f3rd1010111	vd=vs2*vs1 (float)	Vector FP multiply
VMSEQ.VV	R	1010111	000	011000 vm	011000rs2rs1f3rd1010111	vd[i]=(vs2[i]==vs1[i])?1:0	Vector compare to mask
VMERGE.VVM	R	1010111	000	010111 1	010111rs2rs1f3rd1010111	vd[i]=mask?vs1[i]:vs2[i]	Masked merge
VREDSUM.VS	R	1010111	010	000001 vm	000001rs2rs1f3rd1010111	vd[0]=sum(vs2[0..vl-1])	Reduction — sequential dependency